CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 16

File Management and System Call



File Management and System Call

Topics to be Covered
Topics Covered

1. Attributes of File
2. Operations on Files
3. File Accessing Mechanism
4. File Allocation Methods
5. System Calls

1. Attributes of a File

Definition: A file is an **abstract data type** that stores information on secondary storage.

File Attributes:

Attribute	Description
a. Name	Symbolic name of the file; the only human-readable part.
b. Identifier	Unique numeric tag that identifies the file within the file system (non-human-readable).
c. Type	Describes the kind of file (text, executable, binary, etc.). Useful for systems supporting multiple file types.
d. Location	Pointer to the device and the physical address of the file.
e. Size	Current size (in bytes, words, or blocks) and maximum size limit.
f. Time, Date, User ID	Records creation, modification, and access times. Used for **protection and security** purposes.

2. Operations on Files

Since a file is an abstract data type, its operations must be defined.

Main operations include:

1. **Creating a File**
   - Two steps:
     - Find free space in the file system.
     - Create a directory entry for the file (stores file name, location, and other info).
2. **Writing a File**
   - System call specifies the **file name** and the **data to write**.
   - OS locates the file via the directory and maintains a **write pointer** marking the next write position.
   - Pointer is updated after every write.
3. **Reading a File**
   - System call specifies the file name and the **next block** to read into memory.
   - OS locates the file and keeps a **read pointer** marking the next read position.
4. **Repositioning a File (File Seek)**
   - Moves the file pointer to a specific location in the file.
   - Does not involve any actual I/O operation.
5. **Deleting a File**
   - The directory is searched for the file.
   - Space is released, and the directory entry is invalidated.
6. **Truncating a File**
   - Removes file contents but keeps its attributes (useful for reusing the same file structure).

3. File Accessing Mechanisms

Three main mechanisms exist for file access:

1. Sequential Access

- Simplest method.
- Information is processed in order, one record after another (like reading a tape).
- **Read** operation → reads next record and advances file pointer.
- **Write** operation → appends data to the end and advances the pointer.
- Suitable for editors and compilers.

2. Direct Access (Random Access)

- File viewed as a **numbered sequence of blocks**.
- Data can be read/written in **any order** (no restriction).
- Ideal for **databases** where direct retrieval is required.
- Each block is accessed using a **block number parameter** (relative to the start of the file).

3. Indexed Access

- Creates an **index** of key fields with **pointers** to data blocks.
- First, the index is searched → pointer leads directly to the required record.
- Supports efficient direct access in large files.

4. File Allocation Methods

Definition

- File allocation = deciding **how disk blocks are assigned** to files for storage.
- The allocation strategy affects **space efficiency** and **speed of access**.

Factors Affecting Allocation Strategy

1. Speed of sequential and random access.
2. Ease of allocation/deallocation.
3. Support for multi-track and multi-sector transfer.
4. Disk space utilization.
5. Main memory requirement.

Major File Allocation Methods

1. **Contiguous Allocation**
2. **Linked Allocation**
3. **Indexed Allocation**

(a) Contiguous Allocation

- Each file occupies a **set of consecutive blocks** on the disk.
- Disk addresses form a **linear order**.
- Supports both **sequential** and **direct** access.
- **Advantages:**
  - Fast access; easy to compute block addresses.
- **Disadvantages:**
  - **External fragmentation**.
  - Difficulty in file expansion.

Dynamic Storage Allocation (Application of Contiguous Allocation)

- Disk = a large array of blocks divided into **free** and **used** segments.
- Free segment (unallocated) = **hole**.
- Problem: How to allocate memory from holes for a file of given size.
- **Algorithms used:**
  - **First Fit** – Allocate the first hole big enough.
  - **Best Fit** – Allocate the smallest hole that fits.
  - **Worst Fit** – Allocate the largest available hole.

Comparison

Algorithm	Description	Comment
First Fit	Picks the first hole large enough (simple and fast).	May cause small scattered holes.
Best Fit	Picks the smallest sufficient hole.	Leaves the smallest leftover gaps.
Worst Fit	Picks the largest available hole.	Leaves the largest leftover gap; often the least efficient.

- **Note:** First Fit and Best Fit generally perform better than Worst Fit.
- All three suffer from **external fragmentation**.

(b) Linked Allocation

- Each file is a **linked list of disk blocks** scattered over the disk.
- Each block contains a **pointer** to the next block.
- Eliminates **external fragmentation**.
- Supports only **sequential access**, not direct access.
- **Problem:** if a pointer is lost or corrupted, → file becomes inaccessible (reliability issue).

(c) Indexed Allocation

- All pointers to file blocks are kept together in a **single index block**.
- Direct access supported.
- No external fragmentation (any free block can be used).
- Requires an **index table** for every file.
- Suitable for large files.

5. System Calls

Definition

- A **System Call** provides the interface between a **user program** and the **operating system kernel**.
- It allows user mode → kernel mode transition.
- All privileged OS functionalities are accessed through system calls.

Examples

- Accessing files, hardware devices, printers, etc., requires kernel help via **system calls**.
- User-level functions like printf() and scanf() are **library functions** that internally use system calls (not system calls themselves).

Categories of System Calls

Category	Purpose / Examples
1. File Related	Access, create, read, write, or close files. Examples: open(), read(), write(), close(), create()
2. Device Related	Access devices like a printer, monitor, etc. Examples: read(), write(), reposition()
3. Information Related	Get metadata or process information. Examples: getpid() (process ID), getppid() (parent process ID), get time/date.
4. Process Control	Create or terminate processes. Examples: load(), execute(), abort(), fork(), wait(), signal(), allocate()
5. Communication	Handle inter-process communication (IPC). Examples: pipe(), shmget() (shared memory), connect/disconnect client.

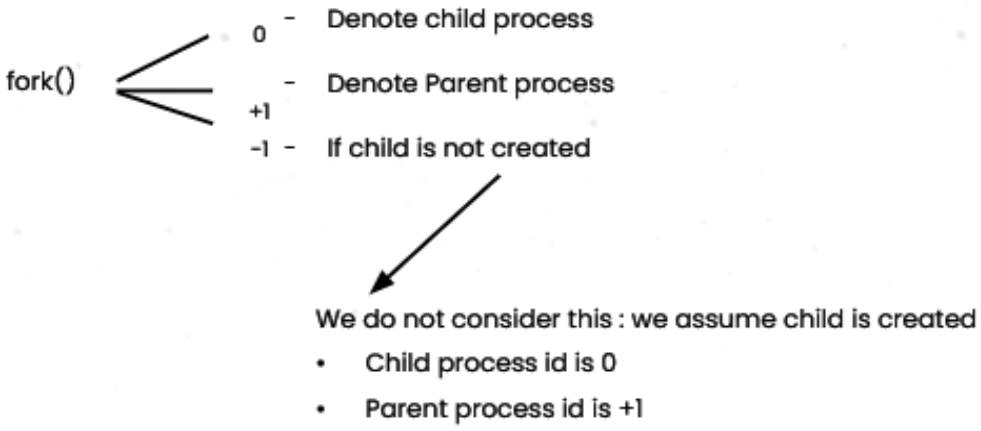
Most Important System Call – fork()

Purpose

- Used to create a **child process** from a **parent process**.
- Enables **multiprocessing** — both parent and child execute concurrently.

Working

- Parent process calls fork().
- OS creates an identical **child process**.
- **Return Values:**
  - **0** → child process
  - **Positive (PID)** → parent process
  - **–1** → error (child not created)



Summary

Concept	Summary
File Attributes	Name, Identifier, Type, Location, Size, Time/Date/User ID
File Operations	Create, Read, Write, Seek, Delete, Truncate
File Access Types	Sequential, Direct, Indexed
File Allocation	Contiguous, Linked, Indexed
Dynamic Storage Allocation	Uses First Fit, Best Fit, Worst Fit
System Call	The interface between the user and kernel modes
Categories	File, Device, Info, Process, Communication
Important Call	fork() creates a child process for multiprocessing

फ़ाइल मैनेजमेंट और सिस्टम कॉल

कवर किए जाने वाले विषय

1. फ़ाइल की विशेषताएँ
2. फ़ाइलों पर ऑपरेशन
3. फ़ाइल एक्सेस करने की प्रक्रिया
4. फ़ाइल एलोकेशन विधियाँ
5. सिस्टम कॉल

1. किसी फ़ाइल के गुण

परिभाषा: फ़ाइल एक अमूर्त डेटा प्रकार है जो सेकेंडरी स्टोरेज पर जानकारी संग्रहीत करती है।

फ़ाइल विशेषताएँ:

विशेषता	विवरण
a. नाम (Name)	फ़ाइल का प्रतीकात्मक नाम; केवल यही भाग मनुष्य द्वारा पढ़ा जा सकता है।
b. पहचानकर्ता (Identifier)	फ़ाइल सिस्टम के भीतर फ़ाइल की पहचान करने वाला अद्वितीय संख्यात्मक टैग (मानव-पठनीय नहीं)।
c. प्रकार (Type)	यह फ़ाइल के प्रकार (टेक्स्ट, एक्ज़ीक्यूटेबल, बाइनरी, आदि) का वर्णन करता है। यह उन सिस्टमों के लिए उपयोगी है जो कई प्रकार की फ़ाइलों का समर्थन करते हैं।
d. स्थान (Location)	डिवाइस का पॉइंटर और फ़ाइल का भौतिक पता।
e. आकार (Size)	वर्तमान आकार (बाइट्स, वर्ड्स या ब्लॉक्स में) और अधिकतम आकार सीमा।
f. समय, दिनांक, यूजर आईडी (Time, Date, User ID)	रिकॉर्ड निर्माण, संशोधन और पहुंच समय को रिकॉर्ड करता है। इसका उपयोग सुरक्षा और संरक्षा उद्देश्यों के लिए किया जाता है।

2. फ़ाइलों पर संचालन

क्योंकि फ़ाइल एक एब्सट्रेक्ट डेटा प्रकार है, इसलिए इसके संचालन को परिभाषित करना आवश्यक है।

मुख्य संचालन में शामिल हैं:

1. एक फ़ाइल बनाना

- **दो चरण:**
  - फ़ाइल सिस्टम में खाली जगह ढूंढें।
  - फ़ाइल के लिए एक डायरेक्टरी बनाएं (इसमें फ़ाइल का नाम, स्थान और अन्य जानकारी स्टोर होती है)।

2. फ़ाइल लिखना

- सिस्टम कॉल फ़ाइल का नाम और लिखने के लिए डेटा निर्दिष्ट करता है।
- ऑपरेटिंग सिस्टम डायरेक्टरी के माध्यम से फ़ाइल का पता लगाता है और अगली लेखन स्थिति को चिह्नित करने वाला एक राइट पॉइंटर बनाए रखता है।
- प्रत्येक लेखन के बाद पॉइंटर अपडेट किया जाता है।

3. एक फ़ाइल पढ़ना

- सिस्टम कॉल फ़ाइल का नाम और मेमोरी में पढ़ने के लिए अगले ब्लॉक को निर्दिष्ट करता है।
- ऑपरेटिंग सिस्टम फ़ाइल का पता लगाता है और अगले पढ़ने की स्थिति को चिह्नित करने वाला एक रीड पॉइंटर रखता है।

4. **फ़ाइल की स्थिति बदलना (फ़ाइल सीक)**
   - यह फ़ाइल पॉइंटर को फ़ाइल में किसी विशिष्ट स्थान पर ले जाता है।
   - इसमें कोई वास्तविक इनपुट/आउटपुट ऑपरेशन शामिल नहीं है।
5. **फ़ाइल हटाना**
   - डायरेक्टरी में फ़ाइल खोजी जाती है।
   - जगह खाली कर दी जाती है और डायरेक्टरी में मौजूद फ़ाइल को रद्द कर दिया जाता है।
6. **फ़ाइल को छोटा करना**
   - यह फ़ाइल की सामग्री को हटा देता है लेकिन उसके गुणों को बरकरार रखता है (समान फ़ाइल संरचना का पुन: उपयोग करने के लिए उपयोगी)।

3. फ़ाइल एक्सेस करने के तंत्र

फ़ाइल तक पहुँचने के लिए तीन मुख्य तंत्र मौजूद हैं:

1. **सीक्वेंशियल एक्सेस**
   - सबसे सरल विधि।
   - जानकारी को क्रमानुसार, एक के बाद एक रिकॉर्ड के हिसाब से संसाधित किया जाता है (जैसे टेप पढ़ना)।
   - पढ़ने की प्रक्रिया → अगला रिकॉर्ड पढ़ता है और फ़ाइल पॉइंटर को आगे बढ़ाता है।
   - लिखने की प्रक्रिया → अंत में डेटा जोड़ता है और पॉइंटर को आगे बढ़ाता है।
   - एडिटर और कंपाइलर के लिए उपयुक्त।
2. **डायरेक्ट एक्सेस (रैंडम एक्सेस)**
   - फ़ाइल को क्रमांकित ब्लॉक अनुक्रम के रूप में देखा जाता है।
   - डेटा को किसी भी क्रम में पढ़ा/लिखा जा सकता है (कोई प्रतिबंध नहीं)।
   - यह उन डेटाबेस के लिए आदर्श है जहाँ सीधे डेटा प्राप्त करना आवश्यक है।
   - प्रत्येक ब्लॉक को ब्लॉक संख्या पैरामीटर (फ़ाइल की शुरुआत के सापेक्ष) का उपयोग करके एक्सेस किया जाता है।
3. **इंडेक्स्ड एक्सेस**
   - डेटा ब्लॉकों के पॉइंटर्स के साथ कुंजी फ़ील्ड्स का एक इंडेक्स बनाता है।
   - सबसे पहले, इंडेक्स में खोज की जाती है → पॉइंटर सीधे आवश्यक रिकॉर्ड तक ले जाता है।
   - बड़ी फ़ाइलों में कुशल प्रत्यक्ष पहुँच का समर्थन करता है।

4. फाइल एलोकेशन मेथड

परिभाषा

- फ़ाइल एलोकेशन = यह तय करना कि फ़ाइलों को संग्रहण के लिए डिस्क ब्लॉक कैसे आवंटित किए जाएंगे।
- एलोकेशन (एलोकेशन) रणनीति स्थान दक्षता और पहुँच की गति को प्रभावित करती है।

एलोकेशन विधि को प्रभावित करने वाले कारक

1. सीक्वेंशियल (अनुक्रमिक) और रेंडम एक्सेस (यादृच्छिक पहुँच) की गति।
2. एलोकेशन/डी-एलोकेशन में आसानी।
3. मल्टी-ट्रैक और मल्टी-सेक्टर स्थानांतरण के लिए समर्थन।
4. डिस्क स्थान उपयोग।
5. मेन मेमोरी की आवश्यकता।

प्रमुख फ़ाइल एलोकेशन विधियाँ

1. **कन्टिगुअस एलोकेशन**
2. **लिंक्ड एलोकेशन**
3. **इंडेक्स एलोकेशन**

(a) कन्टिगुअस एलोकेशन

- प्रत्येक फ़ाइल डिस्क पर लगातार ब्लॉकों का एक समूह घेरती है।
- डिस्क पते एक रैखिक क्रम में होते हैं।
- यह अनुक्रमिक और प्रत्यक्ष दोनों प्रकार की पहुँच का समर्थन करता है।
- **लाभ:**
  - तेज़ पहुँच; ब्लॉक पतों की गणना करना आसान।
- **हानि:**
  - एक्सटर्नल फ्रेगमेंटेशन
  - फाइल के विस्तार में कठिनाई।

डायनेमिक स्टोरेज एलोकेशन (एप्लीकेशन ऑफ कांटिगुअस एलोकेशन)

- डिस्क = ब्लॉकों की एक विशाल सरणी, जो खाली और उपयोग किए गए खंडों में विभाजित होती है।
- फ्री सेगमेंट (अनअलोकेटेड) = रिक्त स्थान (होल)।
- समस्या: दी गई आकार की फ़ाइल के लिए रिक्त स्थानों से मेमोरी कैसे आवंटित करें?
- **उपयोग किए गए एल्गोरिथम:**
  - **फर्स्ट फिट** - सबसे पहले पर्याप्त आकार का छिद्र (होल) आवंटित करें।
  - **बेस्ट फिट** - सबसे छोटा छिद्र (होल) आवंटित करें जो उपयुक्त हो।
  - **वर्स्ट फिट** - उपलब्ध सबसे बड़ा छिद्र (होल) आवंटित करें।

तुलना

एल्गोरिथम	विवरण	टिप्पणी
फर्स्ट फिट	यह पर्याप्त बड़े पहले छिद्र (होल) का चयन करता है (सरल और तेज़)।	इससे छोटे-छोटे अस्तव्यस्त छिद्र (होल) हो सकते हैं।
बेस्ट फिट	सबसे छोटे उपयुक्त छिद्र (होल) का चयन करता है।	इससे बहुत कम अंतराल बचता है।
वर्स्ट फिट	सबसे बड़े उपलब्ध छिद्र (होल) का चयन करता है।	इससे सबसे बड़ा शेष अंतराल रह जाता है; और अक्सर यह सबसे कम कुशल होता है।

- **नोट:** फर्स्ट फिट और बेस्ट फिट आमतौर पर वर्स्ट फिट से बेहतर प्रदर्शन करते हैं।
- तीनों में एक्सटर्नल फ़्रेग्मेंटेशन की समस्या होती है।

(b) लिंक्ड एलोकेशन

- प्रत्येक फ़ाइल डिस्क पर बिखरे हुए डिस्क ब्लॉकों की एक लिंक्ड लिस्ट होती है।
- प्रत्येक ब्लॉक में अगले ब्लॉक का पॉइंटर होता है।
- यह एक्सटर्नल फ़्रेग्मेंटेशन को समाप्त करता है।
- यह केवल सीक्वेंशियल पहुँच का समर्थन करता है, प्रत्यक्ष पहुँच का नहीं।
- **समस्या:** यदि कोई पॉइंटर खो जाता है या दूषित हो जाता है, तो फ़ाइल पहुँच से बाहर हो जाती है (विश्वसनीयता संबंधी समस्या)।

(c) इंडेक्स एलोकेशन

- सभी फ़ाइल ब्लॉकों के पॉइंटर एक ही इंडेक्स ब्लॉक में रखे जाते हैं।
- सीधा एक्सेस समर्थित है।
- कोई एक्सटर्नल फ़्रेग्मेंटेशन नहीं (कोई भी खाली ब्लॉक उपयोग किया जा सकता है)।
- प्रत्येक फ़ाइल के लिए एक इंडेक्स टेबल आवश्यक है।
- बड़ी फ़ाइलों के लिए उपयुक्त।

5. सिस्टम कॉल

परिभाषा

- सिस्टम कॉल उपयोगकर्ता प्रोग्राम और ऑपरेटिंग सिस्टम कर्नेल के बीच इंटरफ़ेस प्रदान करता है।
- यह उपयोगकर्ता मोड से कर्नेल मोड में संक्रमण की अनुमति देता है।
- सभी विशेषाधिकार प्राप्त ऑपरेटिंग सिस्टम कार्यक्षमताओं को सिस्टम कॉल के माध्यम से एक्सेस किया जाता है।

उदाहरण

- फ़ाइलों, हार्डवेयर उपकरणों, प्रिंटरों आदि तक पहुँचने के लिए सिस्टम कॉल के माध्यम से कर्नेल की सहायता की आवश्यकता होती है।
- printf() और scanf() जैसे उपयोगकर्ता-स्तरीय फ़ंक्शन लाइब्रेरी फ़ंक्शन हैं जो आंतरिक रूप से सिस्टम कॉल का उपयोग करते हैं (स्वयं सिस्टम कॉल नहीं)।

सिस्टम कॉल की श्रेणियाँ

श्रेणी	उद्देश्य/उदाहरण
1. फ़ाइल संबंधित	फ़ाइलों तक पहुंचें, बनाएं, पढ़ें, लिखें या बंद करें। उदाहरण: open(), read(), write(), close(), create()
2. डिवाइस संबंधी	प्रिंटर, मॉनिटर आदि जैसे उपकरणों का उपयोग करें। उदाहरण: read(), write(), reposition()
3. जानकारी संबंधी	मेटाडेटा या प्रक्रिया संबंधी जानकारी प्राप्त करें। उदाहरण: getpid() (process ID), getppid() (parent process ID), समय/तारीख प्राप्त करें।
4. प्रक्रिया नियंत्रण	प्रक्रियाएँ बनाएँ या समाप्त करें। उदाहरण: load(), execute(), abort(), fork(), wait(), signal(), allocate()
5. संचार	अंतर-प्रक्रिया संचार (IPC) को संभालें। उदाहरण: pipe(), shmget()(शेयर मेमोरी), क्लाइंट को कनेक्ट/डिस्कनेक्ट करें।

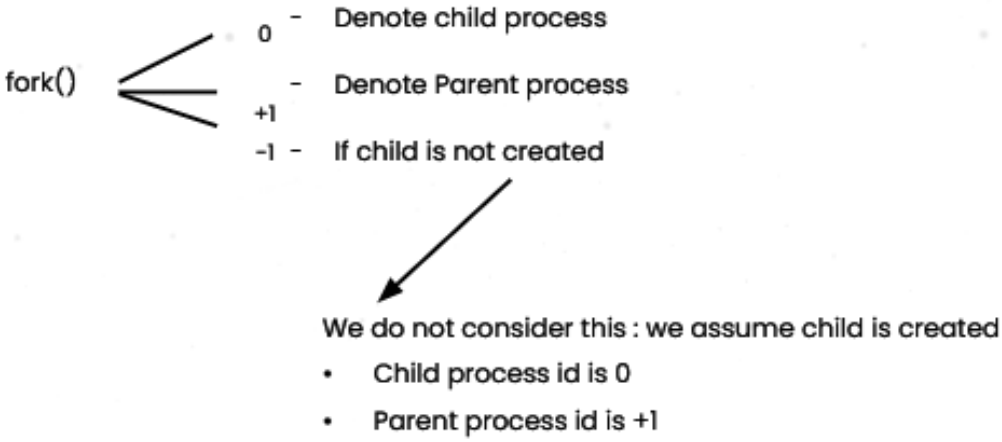
सबसे महत्वपूर्ण सिस्टम कॉल – fork()

उद्देश्य

- पैरेंट प्रोसेस से चाइल्ड प्रोसेस बनाने के लिए उपयोग किया जाता है।
- यह मल्टीप्रोसेसिंग को सक्षम बनाता है — पैरेंट और चाइल्ड दोनों एक साथ निष्पादित होते हैं।

कार्यरत

- पैरेंट प्रोसेस fork() फ़ंक्शन को कॉल करता है।
- OS एक समान चाइल्ड प्रोसेस बनाता है।
- **रिटर्न वैल्यू:**
  - 0 → चाइल्ड प्रोसेस
  - पॉजिटिव (PID) → पैरेंट प्रोसेस
  - –1 → त्रुटि (चाइल्ड प्रोसेस नहीं बनाया गया)



सारांश

अवधारणा	सारांश
फाइल एट्रीब्यूट्स	नाम (Name), पहचानकर्ता (Identifier), प्रकार (Type), स्थान (Location), आकार (Size), समय/दिनांक/उपयोगकर्ता आईडी (Time/Date/User ID)
फाइल आपरेशंस	क्रिएट (Create), रीड (Read), राइट (Write), सीक (Seek), डिलीट (Delete), ट्रंकेट (Truncate)
फाइल एक्सेस टाइप्स	सीक्वेंशियल (Sequentia), डायरेक्ट (Direct), इंडेक्स्ड (Indexed)
फाइल एलोकेशन	कांटिगुअस (Contiguous), लिंक्ड (Linked), इंडेक्स्ड (Indexed)
डायनामिक स्टोरेज एलोकेशन	फर्स्ट फिट, बेस्ट फिट, वर्स्ट फिट का उपयोग करता है।
सिस्टम कॉल	उपयोगकर्ता और कर्नेल मोड के बीच इंटरफ़ेस
श्रेणियाँ (कैटेगरीज़)	फ़ाइल, डिवाइस, जानकारी, प्रक्रिया, संचार
इंपोर्टेंट कॉल	fork() फ़ंक्शन मल्टीप्रोसेसिंग के लिए एक चाइल्ड प्रोसेस बनाता है।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if